SuperAI SS6

An Autonomous AI Software-Engineering Pipeline, Validated on a Production-Shaped Mock Application

Design, Evaluation, Live Production Validation, and a Path to Real-World Adoption

Technical Report - eleven-7 Case Study

Version 1.0 · June 2026

Contents

Abstract

This report presents SuperAI SS6, an autonomous software-engineering pipeline that ingests a natural-language requirement and carries it through five phases - Understand, Plan, Debate, Execute, and Review - before halting for human approval. The pipeline combines local retrieval-augmented generation (RAG) for codebase grounding, a provider-agnostic large language model (LLM) layer, a deterministic weighted evaluator for selecting among competing implementation plans, and a context-aware compliance gate that enforces a project's architectural rules. To evaluate the system under realistic conditions, we constructed eleven-7, a production-shaped mock of a convenience-store delivery platform modelled on Thailand's 7-Eleven delivery service, comprising an Angular front end, a Node.js/Express API, a MySQL data model, and AWS SNS/SQS/SMS messaging emulated with LocalStack. We report results across four evaluation harnesses, a unit-test suite, and - critically - a live end-to-end deployment using Docker on real infrastructure, during which the system surfaced and we corrected six integration defects that static, offline testing could not detect. We additionally repackage the pipeline as a pip-installable library exposing both a command-line tool (ss6) and a callable Python API. We conclude with an impact analysis, representative real-world use scenarios, a production-readiness gap assessment, and a roadmap toward operational deployment.

Keywords: *autonomous software engineering, retrieval-augmented generation, LLM agents, human-in-the-loop, compliance gating, microservices, reproducible evaluation.*

1. Introduction

Large language models have made it feasible to automate substantial portions of the software-development lifecycle. However, moving from a compelling demonstration to a tool a team can trust requires three properties that are frequently absent: grounding in the actual target codebase, adherence to a project's architectural and stylistic rules, and a safety model that keeps a human in control of what is merged. SuperAI SS6 (SS6) is an attempt to satisfy these properties in a small, reproducible, and inspectable system.

The pipeline is organised around five phases. (1) Understand retrieves the files and rules most relevant to a requirement using local RAG. (2) Plan produces three deliberately distinct implementation plans - optimised for performance, for component reuse and maintainability, and for speed of delivery. (3) Debate scores those plans against project priorities using a transparent, deterministic weighting scheme and selects a winner. (4) Execute implements the winning plan on an isolated Git branch, never touching the working tree. (5) Review runs an automated compliance suite derived from the project's System Blueprint and then halts, presenting a diff for human approval rather than auto-merging.

Our contributions are: (i) a complete, offline-capable implementation of the five-phase pipeline with one evaluation harness per phase; (ii) eleven-7, a realistic, internally consistent mock application used as the target codebase and test bed; (iii) a live production validation on real Docker/MySQL/LocalStack infrastructure, including a catalogue of integration defects discovered only at runtime; and (iv) a packaging effort that turns the research prototype into a pip-installable tool with a CLI and a programmatic API.

2. Background and Related Work

Retrieval-augmented generation. RAG conditions a generative model on documents fetched from an external store, improving factual grounding and reducing hallucination [1]. SS6 applies this to source code: each requirement is embedded and matched against a chunked index of the target repository and its rule file, so plans and code reference real, resolvable files. Embeddings use Sentence-BERT-style encoders [2]; when unavailable the system degrades to a deterministic lexical fallback.

Agent orchestration. The phases are modelled as a state graph in the style of LangGraph, where a typed state object is read and written by successive nodes [3], keeping each phase independently testable and the control flow explicit.

Human-in-the-loop and safety. Rather than auto-merging generated code, SS6 follows a human-in-the-loop discipline: Execute writes to an isolated branch and Review halts for approval, mirroring established guidance that autonomous changes to production systems must remain gated by human judgement.

Twelve-factor readiness. The eleven-7 target application follows twelve-factor conventions [4] - configuration from the environment, stateless processes, explicit backing services - which is what allowed it to be containerised and validated end to end.

3. System Architecture

3.1 The SS6 pipeline

The pipeline is a Python package, agent_pipeline, of roughly 1,900 lines. Each phase owns an agent and an output artifact, and each is paired with an evaluation harness so that quality can be measured rather than asserted.

Phase	Owner	Output	Evaluation
1. Understand	RAG / Retriever	ranked files + rules	Recall@k, MRR
2. Plan	Architect	three plans (A/B/C)	schema, distinctiveness, grounding
3. Debate	Evaluator	scored winner	weight-sensitivity
3b. Execute	Developer	code on isolated branch	compliance gate
4. Review	Compliance + HITL	review packet, halt	gate discrimination

Provider-agnostic reasoning. The LLM layer resolves, in order, to Anthropic Claude, Google Gemini, a local Ollama model, or a deterministic mock. The mock templates output from injected grounding facts so the entire pipeline and all evaluations run at zero cost and with no network access - essential for reproducibility and privacy-sensitive settings.

The Evaluator is intentionally not an LLM. Plan selection is pure Python: each plan receives sub-scores on reuse, blueprint adherence, performance, and speed, combined with project-priority weights. This makes the most consequential decision - which plan to build - deterministic, explainable, and identical across providers.

3.2 The eleven-7 target application

eleven-7 is a mock convenience-store delivery platform of 87 files and approximately 3,830 lines, modelled on the real 7-Eleven delivery service in Thailand (delivery from the nearest branch; TrueMoney/PromptPay/card/cash payments; an ALL Member loyalty programme) [5][6]. It is a monorepo with a clear separation of concerns:

- **Front end:** Angular 17 ops console built from five canonical shared primitives (Card, Badge, MetricTile, Button, Avatar) and design tokens.
- **Back end:** Node.js/Express + TypeScript REST API with a repository data layer, pure-function services, validated controllers, and SQS worker consumers.
- **Data:** MySQL 8 with all money stored as integer minor units (satang); subtotals, totals, payments, point accruals, and refunds reconcile exactly.
- **Messaging:** AWS SNS/SQS/SMS via the v3 SDK, emulated locally with LocalStack, plus an in-process fake mode for offline runs.

3.3 Branded internal services

Each domain service carries a product codename declared in a registry and surfaced at a /services endpoint: ShelfScan (catalog), PricePilot (pricing), OrderForge (orders), FleetDash (delivery), PulseNotify (notifications), PaySwift (payments), PerksEngine (promotions and

points), CareDesk (returns and support), and StockKeeper (inventory). The order lifecycle - pending, confirmed, preparing, dispatched, delivered - is driven asynchronously through SQS with SMS notifications at each transition.

4. Methodology

We evaluate SS6 along two independent axes. The first is offline correctness: the pipeline and its four harnesses run deterministically with the mock provider and the lexical RAG fallback, establishing that the wiring, gates, and scoring behave as specified. The second is live integration: the eleven-7 stack is built and run on real Docker infrastructure with a real MySQL database and LocalStack-emulated AWS, exercising the paths that offline testing cannot reach - container builds, schema loading, cross-service networking, and asynchronous message processing.

Reproducibility. All offline results use the mock provider and a lexical embedder, so they are bit-for-bit reproducible. Live results used the pinned images mysql:8.0 and localstack/localstack:3 and Node 20 runtime images.

Data-integrity validation. Before any execution the seed data was checked programmatically for referential integrity, enumerated-value domains, uniqueness, address-to-customer ownership, and money arithmetic across all four migrations; all checks passed.

5. Evaluation and Results

5.1 Phase-level evaluation (offline)

All four phase harnesses and the unit-test suite pass against the eleven-7 corpus (76 indexed files, 155 chunks).

Harness	What it verifies	Result
Design quality	completeness, typed UML, >=6 test cases, grounding	**PASS**
Plan quality	schema, archetype coverage, distinctiveness (sim 0.10), grounding	**PASS**
Debate	winner correct under 4 weight profiles	**PASS**
Execution gate	good passes; 5 violation fixtures caught	**PASS**
Unit tests (pytest)	RAG, normalization, evaluator, developer, debate	**17 passed**
Backend pure logic	pricing, money, ETA, coupons, points, refunds	**43 assertions**

Debate behaviour. Under the default priorities (reuse 0.40, blueprint 0.30, performance 0.20, speed 0.10) the evaluator selects the reuse-oriented Plan B with a margin of 0.17. A weight-sensitivity test confirms soundness: a performance-dominant profile elects Plan A, a speed-dominant profile elects Plan C, and a reuse-dominant profile re-elects Plan B - each archetype wins exactly when its dimension dominates.

Compliance-gate discrimination. A gate that always passes is worthless. The Review suite admits a compliant Angular component and rejects five non-compliant fixtures: an inline hex colour, a direct fetch() call, HttpClient inside a component, a screen reusing no canonical primitive, and a file with no export.

5.2 Retrieval quality

With the semantic encoder (all-MiniLM-L6-v2) and a persistent vector store, the hardened retrieval set lands every positive in the top three (Recall@3 = 100%, MRR ~ 0.96) on the reference corpus. On a host without the encoder, the deterministic lexical fallback is used; non-paraphrased queries still retrieve the correct file at rank 1, while deliberately paraphrased queries miss - the expected behaviour that makes the metric meaningful. The semantic baseline for the eleven-7 corpus should be re-measured on a host with the encoder present; the lexical fallback verifies plumbing, not model quality.

5.3 Live production validation

The full stack was built and run with Docker on real infrastructure. MySQL applied all four migrations on first boot; the API served live data; the asynchronous order pipeline completed end to end.

Check	Observation
Containers	api, worker, mysql (healthy), localstack (healthy), frontend - all up
Database	all 4 migrations loaded; cross-table SQL (revenue, refunds, low-stock) correct
Revenue query	11 non-cancelled orders, gross THB 3,008.00
API + UI	all 12 endpoints and the Angular app return HTTP 200
Order flow	place, confirmed, dispatched, courier assigned, two SMS sent
Payments	valid capture/refund succeed; invalid attempts return 422 (not 500)
Error budget	0 unhandled API errors, 0 worker failures

5.4 Defects surfaced only by live testing

The live run is the centrepiece of this evaluation because it exposed six integration defects that the offline sandbox could not, each corrected and re-verified - direct evidence for the value of real-infrastructure validation over static checking.

#	Defect	Root cause	Fix
1	Back end did not compile	mysql2 typings reject named-parameter objects	explicit type casts in the data layer
2	No SNS topic / queues	LocalStack init script not executable when mounted	made the script executable
3	Worker crashed on boot	ran a dev-only tool absent from the prod image	run the compiled JavaScript
4	Malformed job messages	work queue subscribed to the event topic	separate queue/topic; guard the consumers

#	Defect	Root cause	Fix
5	Mojibake (All Cafe)	loader client defaulted to latin1	SET NAMES utf8mb4 in migrations
6	Rule violations returned 500	service threw a generic error	domain error mapped to HTTP 422

6. Impact Analysis

Engineering productivity and consistency. SS6 turns a one-line requirement into a grounded design, three costed options, a justified decision, and a compliant, review-ready change. The deterministic debate gives teams an auditable rationale for the chosen approach, and the compliance gate mechanically enforces conventions otherwise left to reviewer vigilance.

Safety and governance. Because the system never merges and confines all work to an isolated branch, the blast radius of a bad generation is zero; the human-in-the-loop halt is structural, not advisory.

Cost and privacy. The offline mock and lexical fallback let the pipeline run at zero marginal cost with no data leaving the machine - relevant for education, budget-constrained teams, and regulated environments. A live provider is enabled with a single environment variable when higher-quality reasoning is warranted.

Educational value. The project is a compact, end-to-end illustration of RAG, agent orchestration, deterministic evaluation, and human-in-the-loop safety, paired with a realistic multi-tier application.

7. Real-World Use Scenarios

1. **Feature-request triage.** A product owner files add ALL Member points redemption at checkout. SS6 retrieves PerksEngine and the pricing service, proposes three approaches, selects the reuse-oriented one, and produces a branch plus a review packet - the engineer starts from a compliant draft rather than a blank file.
2. **Convention enforcement at scale.** An organisation encodes its rules once in the System Blueprint; every generated change is then checked automatically, catching inline colours, direct network calls, or missing primitive reuse before human review.
3. **Onboarding and exploration.** A new engineer asks the retriever where a behaviour lives (how is the delivery fee computed?) and receives the exact files, accelerating ramp-up.
4. **Architecture-decision support.** By re-running the debate with different priority weights, a tech lead sees how the recommendation shifts as the team re-prioritises performance versus speed versus maintainability.
5. **Cost-tiered operation.** Routine changes run on a free local model or the mock; high-stakes changes are escalated to a frontier model - the same pipeline, a different provider.

8. Production-Readiness Assessment and Roadmap

We distinguish what is already production-grade from what remains an MVP concern, and propose a roadmap. The packaging in Section 9 directly addresses adoption friction.

Area	Status	Gap / next step
Pipeline correctness	Strong	deterministic, fully tested; re-measure semantic Recall@k on an encoder host
Safety model	Strong	isolated branch + HITL halt; add an approver policy
Packaging / UX	Strong	pip install + ss6 CLI + Python API delivered
Target-app runtime	Adequate	runs via Docker; add CI, a migration tool, a secrets manager
Observability	MVP	structured logs only; add metrics, tracing, readiness probes
LLM plan quality	Unproven	gates verified on mock; human spot-check needed on a live provider
Security	MVP	mock auth; add real authn/z, rate limiting, dependency scanning

Roadmap. (1) Re-run the semantic retrieval baseline and a live-provider plan-quality spot-check. (2) Add CI that runs the four harnesses and the unit tests on every change. (3) Harden the target app for operations: readiness/liveness probes, a migration runner, metrics and tracing, real authentication. (4) Extend the compliance gate with project-specific rules and a severity model. (5) Publish the package to an internal index for one-command installation.

9. Packaging for Adoption

To make the pipeline easy to implement in practice, it is distributed as a standard Python package (PEP 621 pyproject.toml). Installation is a single command, and the tool is usable both from the shell and as a library, so it can be embedded in other agents or automation.

Command line. ss6 rag, ss6 plan, ss6 debate, ss6 execute, ss6 run, and ss6 eval expose the phases directly; warnings are routed to stderr so machine-readable JSON on stdout stays clean and composable.

Python API. from agent_pipeline import retrieve, plan, debate, execute, run - each returns plain dictionaries, making the functions convenient to call as tools. Optional extras pull in heavier dependencies only when needed; the core import works everywhere and degrades gracefully.

Verification. After packaging, pip install -e . succeeds, ss6 run completes the full loop and writes the artifacts, ss6 eval passes, and the unit-test suite remains green - confirming the refactor introduced no regression.

10. Limitations and Threats to Validity

- The plan and code content shown offline are produced by a deterministic mock; while the gates and scoring are provider-independent, the substantive quality of plans and code from a live model has not yet been graded by human reviewers.
- The semantic retrieval baseline for the eleven-7 corpus was not re-measured on a host with the sentence encoder installed; the reported lexical numbers verify plumbing, not model quality.
- eleven-7 is a mock: its data and integrations are realistic but synthetic, and the AWS services are emulated; behaviour against real managed services may differ.
- The compliance gate uses lightweight syntactic checks; it is sound for the encoded rules but not a substitute for full static analysis or tests run inside the target repository.
- Evaluation centres on a single target application; generalisation to other stacks and larger codebases remains to be demonstrated.

11. Conclusion

SuperAI SS6 demonstrates that an autonomous software-engineering pipeline can be grounded, rule-abiding, safe, reproducible, and - after packaging - genuinely easy to adopt. By pairing the pipeline with eleven-7, a production-shaped mock, and validating the whole system on real Docker and MySQL infrastructure, we obtained evidence that static testing alone would have missed: six concrete integration defects, each found and fixed at runtime. The deterministic evaluator and the human-in-the-loop compliance gate keep the consequential decisions transparent and the merge boundary firmly under human control. The remaining gaps - a live-provider quality study, a re-measured semantic baseline, and operational hardening - are well understood and form a clear roadmap toward production use.

References

[1] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS).

[2] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP.

[3] LangChain. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. Documentation, https://langchain-ai.github.io/langgraph/.

[4] Wiggins, A. (2011). The Twelve-Factor App. https://12factor.net/.

[5] Thaiger. How to use the 7-Eleven delivery application (7Delivery). https://thethaiger.com/guides/best-of/lifestyle/how-to-use-7-eleven-delivery-application.

[6] 711 Thai Snacks. 7-Eleven Delivery Thailand: The Ultimate Guide (2026). https://www.711thaisnacks.com/7-eleven-delivery-thailand-app-guide/.